

第 8 章 * 深度学习处理器运算器设计

深度学习处理器 (Deep Learning Processor, DLP) 是一类高效支持深度学习算法的处理器, 其设计目标是针对深度学习算法进行更加高效的处理。DLP 实现高效处理的一项基本原理在于采用了矩阵乘法、矩阵乘向量等高级线性代数运算作为基本算子; 相比于 CPU、GPU 采用的标量、向量运算算子, 这些高级算子具有更高的运算密度, 使处理器在同等访存能力水平下能够具有更强的运算能力。

在当前流行的深度学习大语言模型推理运算过程当中, 矩阵乘向量这种运算模式占总运算量的 90% 以上; 进行多批次推理运算时, 主要运算模式升级为矩阵乘法。大语言模型的运算模式恰好符合 DLP 的设计思想, 因此, DLP 芯片产品发展成为智能计算系统高效处理大语言模型推理运算的首选方案。与此同时, 传统深度学习模型当中常用的图像卷积等运算也可经等价变换后由矩阵乘法运算完成, 同样适合在 DLP 上执行。

本章实验内容旨在介绍 DLP 矩阵运算器的设计方法。本章首先展示一个内积运算器的例子, 通过这个例子介绍实验环境和硬件设计语言 Verilog 的基础知识, 演示如何使用 EDA 工具进行仿真调试; 随后, 实验内容将引导读者自行设计更为复杂的矩阵乘向量运算器、矩阵乘法运算器, 最终掌握 DLP 核心运算部件的基本设计方法。为了让无硬件设计基础的读者也能顺利开展实验, 我们在实验过程中鼓励读者在大语言模型的辅助下尝试完成 Verilog 语言程序的编写。需要说明的是, 本章的实验设计仅为教学原型, 与实际 DLP 芯片产品中所需的设计方案在功能完备性和结构复杂性上仍存在较大差距, 但此原型已经完整体现了 DLP 加速深度学习运算的基本原理。

8.1 实验目的

掌握深度学习处理器中矩阵运算器的设计原理, 能够使用 Verilog 语言实现内积运算器、矩阵乘向量运算器、矩阵乘法运算器的设计并进行功能仿真。具体包括:

- 1) 理解深度学习处理器加速深度学习运算的原理, 理解本实验和深度学习处理器基本模块间的关系。
- 2) 使用 Verilog 语言实现内积运算器, 理解内积运算器的基本结构, 掌握基础的硬件设计流程。
- 3) 在内积运算器的基础上, 使用 Verilog 语言实现矩阵乘向量运算器, 加深对深度学习处理器加速原理的理解。
- 4) 在内积运算器、矩阵乘向量运算器的基础上, 使用 Verilog 语言实现矩阵乘法运算器, 加深对矩阵运算单元的理解。

实验工作量: 约 300 行代码, 约需 4 个小时。

8.2 背景介绍

本节首先介绍分析深度学习算法中的算法特征（包括大语言模型和卷积神经网络），然后介绍深度学习处理器架构，接下来介绍矩阵运算在深度学习处理器上的处理过程，最后介绍本实验环境，包括工具安装和验证环境。

8.2.1 算法特征

最常用的深度学习模型有两类，一类是 Transformer 结构，主要算子是全连接层和注意力层，目前主要应用在自然语言处理任务、较大规模的计算机视觉任务中，例如大语言模型（LLM）；另一类是卷积神经网络结构，主要算子是卷积层和全连接层，目前主要应用在中小规模的计算机视觉任务中，例如图像分类、目标检测等。其中，全连接层可刻画为矩阵乘向量运算，注意力层可刻画为矩阵乘法运算；如果是在自然语言处理的预填充计算阶段，或者在云端推理等追求较高吞吐的应用场景中，这些模型往往会将多次推理任务组合为一个批次一同进行，此时全连接层也将形成矩阵乘法运算。因此，矩阵乘向量、矩阵乘法是深度学习运算中非常基本的运算模式。

卷积层的情况则更复杂一些，我们对它的运算过程进行推导，尝试将它同样归结为矩阵乘法运算。卷积层由输入神经元激活值 $\mathbf{X}$ 、输出神经元激活值 $\mathbf{Y}$ 和权重 $\mathbf{W}$ （即卷积核）组成。输入神经元激活值 $\mathbf{X}$ 各维度的尺寸为 (c_i, h_i, w_i) ，即包含了 c_i 个特征图，每个特征图尺寸为 $h_i \times w_i$ ；输出神经元激活值 $\mathbf{Y}$ 各维度的尺寸为 (c_o, h_o, w_o) ，即包含 c_o 个特征图，每个特征图尺寸为 $h_o \times w_o$ 。相应地，卷积核各维度的尺寸应为 (c_o, c_i, k_h, k_w) ，其中 k_h 和 k_w 分别表示卷积核在空间维度上的高和宽。要得到一个完整的输出特征图，可以取所有输入特征图与权重中对应输出特征图的卷积核（尺寸为 $c_i \times k_h \times k_w$ ），并进行卷积运算。这一过程对每一个输出特征图重复进行，最终可得到所有的输出特征图。

一种实现方式是：取输入特征图在空间维度上 $k_h \times k_w$ 大小的区域内的神经元激活值，依次与卷积核中的对应参数做乘法运算并求和，得到每一个输出特征图上同一位置的神经元激活值；然后让输入特征图的取值区域沿着空间维度方向分别滑动，重复此步骤进行运算，直到得到所有输出特征图的所有神经元激活值。其中，第 n 个输出特征图上空间坐标 (y, x) 位置的神经元激活值的计算公式为

$$Y(n, y, x) = \sigma \left(\sum_{i=0}^{k_w-1} \sum_{j=0}^{k_h-1} \sum_{k=0}^{c_i-1} X(k, j+y, i+x) \times W(k, j, i, n) \right) \quad (8.1)$$

其中， i, j, k 是三个仅用于循环计数的临时变量（注意不要与卷积核尺寸 k_h, k_w 相混淆）， σ 表示激活函数。

注意到三个求和维度可以重整聚合到一起，式(8.1)即可统一表达为内积运算。为此，我们可以重新排列输入神经元激活值（并适当地允许重复的值出现），将卷积运算过程中每一步的取值区域的输入神经元激活值表示为一个向量，所有的这些向量共同组成矩阵 $\bar{\mathbf{X}}$ 。具体而言，组成的矩阵应有 $\bar{\mathbf{X}}(\varphi_1(x, y), \varphi_2(i, j, k)) = \mathbf{X}(k, j+y, i+x)$ 对 x, y, i, j, k 任意（有意义的）取值均成立，其中 φ_1, φ_2 表示将多维度的坐标映射到二维坐标的编码函数，可以选用任何一对紧凑编码的函数，它们决定了数据在存储器中摆放的顺序。例如，人们常用

$\varphi_1(x, y) = w_0 y + x$ 与 $\varphi_2(i, j, k) = k_h k_w k + k_w j + i$ 来实现。这种高维张量到矩阵的变换操作称为图像到列转换 (image to column, 常简记为 `im2col`)。

与此同时, 将 W 相应地改记为 $\overline{W}$, 使得 $\overline{W}(\varphi_2(i, j, k), n) = W(k, j, i, n)$ 。此时, 公式(8.1)可简化为

$$Y(n, y, x) = \overline{Y}(n, \varphi_1(x, y)) = \sum_{i=0}^{c_i k_h k_w - 1} \overline{X}(\varphi_1(x, y), i) \times \overline{W}(i, n) \quad (8.2)$$

更进一步地简化, 以矩阵乘法来表示, 即

$$\overline{Y} = \overline{X} \overline{W} \quad (8.3)$$

由此可知, 卷积层运算可以通过重新排列数据在张量中的位置, 等价变换为矩阵乘法运算。事实上, 深度学习处理器采用了高效的硬件控制结构, 依照此原理对数据流进行控制, 以矩阵运算处理卷积层。这也是无论是在大语言模型为主流的时代、还是卷积神经网络为主流的时代, 深度学习处理器都选择以矩阵运算器作为核心运算部件的原因。

8.2.2 DLP 总体架构

图8.1展示了一个最简单的深度学习处理器 (DLP) 架构。其中, 矩阵运算单元承担了深度学习的运算功能, 是最重要的模块; 运算所需的输入神经元激活值、权重和运算产生的输出神经元激活值分别需要专门的便笺存储器暂时存放在 DLP 内部, 以便尽可能地复用相同的数据; 直接内存访问模块 (DMA) 统一负责数据的搬运, 可以让数据在 DLP 之外的主存储器和 DLP 之内的三块便笺存储器之间按特定的方向流动, 且不干扰处理器其他部分的运行。以上所有模块都通过 DLP 的控制器统一提供控制信号, 控制器所读取的指令也一并通过 DMA 从主存储器中取来。至此, 形成了一个最小可用的 DLP 结构框架。实际的 DLP 结构会在此基础之上增设更多的模块, 例如标量、向量运算器等, 关于它们的作用和结构我们已在理论教材中进行了介绍。本章实验内容主要面向 DLP 架构中最核心、也是最独特的模块——矩阵运算单元的设计和实现。

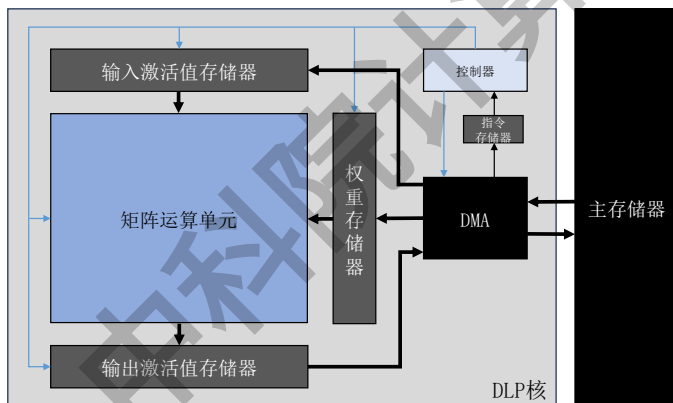


图 8.1 深度学习处理器总体架构

8.3 实验环境

通过 Verilog 语言实现的模块需要使用 HDL 仿真工具进行编译调试。本节使用 Ubuntu 24.04 LTS 版本的操作系统，在该环境上用 Verilator 和 GTKWave 仿真工具来完成实验。

Verilator 是一款开源的 Verilog/SystemVerilog 仿真器和代码生成器。它能够将硬件描述语言 (HDL) 转换成高性能的 C++ 或 SystemC 的库等中间文件，从而完成数字电路的功能仿真和验证。此外，它也具有静态代码分析的功能。

Ubuntu 是一个基于 Linux 内核的开源操作系统，Ubuntu 24.04 LTS 是 Canonical 公司发布的 Ubuntu 操作系统的一个长期支持版本。对于微软 Windows 操作系统用户而言，Ubuntu 24.04 LTS 也可直接在系统内置的微软商店中下载，以子系统的方式运行在 Windows 系统内部。

8.3.1 工具安装

(1) 安装依赖，Verilator 的编译需要安装以下依赖（安装过程中的部分错误可以忽略）：

```
1 sudo apt-get install git help2man perl python3 make autoconf g++ flex bison ccache
2 sudo apt-get install libgoogle-perftools-dev numactl perl-doc
3 sudo apt-get install libfl2 # 仅针对Ubuntu，忽略错误
4 sudo apt-get install libfl-dev # 仅针对Ubuntu，忽略错误
5 sudo apt-get install zlibc zlibg zlibg-dev # 仅针对Ubuntu，忽略错误
```

(2) 下载 Verilator 源码，我们可以通过 `git clone` 命令下载 Verilator 的国内镜像源码（仅第一次需要下载）。执行以下命令：

```
1 git clone https://gitee.com/mirrors/Verilator.git
```

(3) 编译安装，使用 `cd` 命令进入 Verilator 的源码目录文件，执行以下命令：

```
1 autoconf # 创建configure脚本
2 ./configure # 配置Makefile文件
3 make -j `nproc` # 编译Verilator，如果出错可以尝试简化为'make'
4 sudo make install
```

(4) 安装 GTKWave 波形查看工具，执行以下命令：

```
1 sudo apt-get install gtkwave
```

(5) 安装检查，执行以下命令：

```
1 verilator --version
2 gtkwave --version
```

如果输出 Verilator 和 GTKWave 版本号则说明安装成功。

8.3.2 代码文件组织

实验环境文件夹组织如下：

- 目录 `src`，包含编写的实验 Verilog 源代码。
- 目录 `sim`，仿真相关脚本、测试文件及编译产物。
 - 文件 `tb_top.cpp`，测试顶层文件，生成激励信号，实例化矩阵运算子单元模块。
 - 文件 `Makefile`，管理编译流程，定义如何将 Verilog 与 C++ 编译成可执行文件。
 - 文件 `compile.f`，编译文件列表，用于指定编译脚本需要编译的文件。
- 目录 `data`，包含仿真输入输出数据文件。
 - 向量规模描述文件 `config`，描述需进行内积计算的向量规模。
 - 输入神经元文件 `neuron`，存储输入的神经元数据。
 - 输入权重文件 `weight`，存储输入的权重数据。
 - 输出结果文件 `result`，存储运算结果。

8.4 实验内容

在通用处理器中，神经元数据和权重数据一般采用单精度浮点数据表示，相应的运算单元也采用浮点运算器。然而在深度学习处理器中，为了节省功耗、面积开销，会使用低精度数据格式；由于各应用场景的资源预算不同、精度需求不同，人们会混合使用多种低精度数据格式。为了简化实验，本实验实现的运算器的所有神经元/权重数据都采用 16 位有符号整数（INT16）类型表示。

为了便于理解深度学习矩阵运算单元的设计和迭代开发，本实验分为三个步骤，分别实现内积运算器、矩阵乘向量运算器和矩阵乘法运算器。最后介绍如何使用仿真工具对代码进行编译和调试。

8.5 实验步骤

8.5.1 内积运算器

内积运算器的核心功能是实现向量的乘积累加，即输入神经元激活值向量与权重向量逐元素相乘，然后再将所有乘积结果累加到一起。我们以固定向量长度为 4 来展示一个例子。图8.2给出了一个简单实现，它包含输入寄存器、乘法器、加法器和结果寄存器等核心组件。该示例采用同步复位、输入寄存器以及固定的两级流水线（输入采样一拍、计算输出一拍），每个时钟周期可完成一个完整的长度为 4 的内积运算。

示例的内积运算器的输入输出接口信号如表8.1所示。该运算器每个时钟周期可并行接收四个 16 位有符号输入神经元激活值（`activations[0:3]`）和四个 16 位有符号权重（`weights[0:3]`），输出为 32 位有符号内积结果（`result[0:31]`）。核心控制信号包括时钟（`clk`）、同步复位信号（`reset`，高电平有效）和使能信号（`enable`，高电平有效）。采用两级流水线结构。运算器包含两组输入寄存器（`activations_reg` 和 `weights_reg`）用于缓存输入数据，以及一个 32 位结果寄存器（`result`）。当同步复位信号 `reset` 有效时，所有

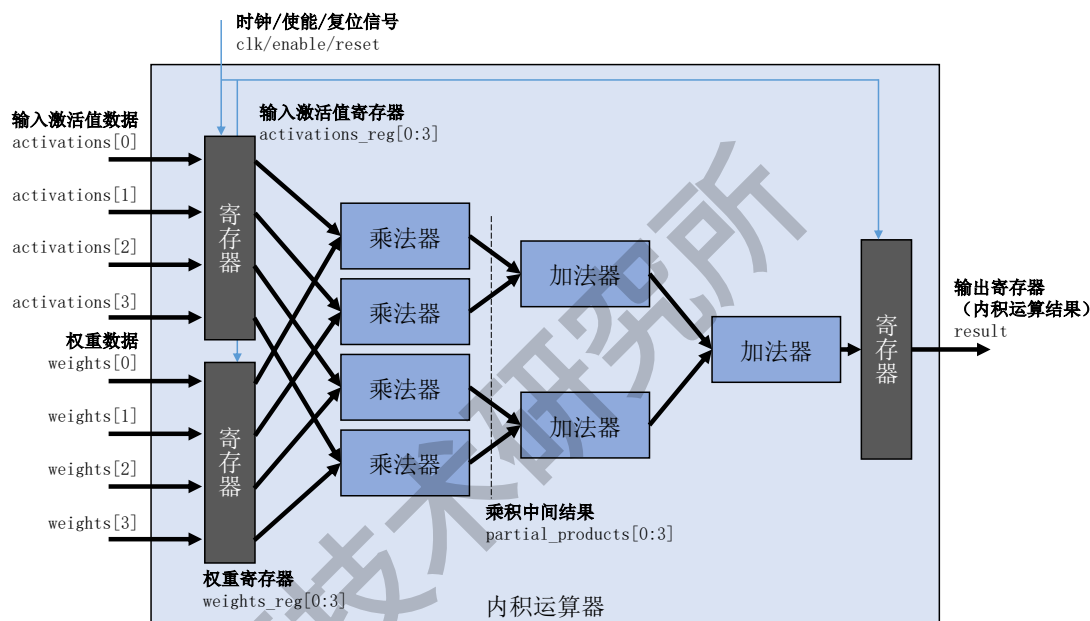


图 8.2 内积运算器

寄存器均被清零；当使能信号 `enable` 有效时，在时钟上升沿触发下，四组神经元激活值和权重数据将同步锁存至对应的输入寄存器。四个并行连接的乘法器可在一个时钟周期内同时完成四对数据的乘法运算，产生四个 32 位乘积中间结果 (`partial_products[0:3]`)。这些乘积中间结果会在当前时钟周期即通过加法树完成累加。在下一个时钟上升沿，如果使能信号 `enable` 有效，累加结果将被锁存至输出寄存器，输出到内积运算器模块之外。

需要注意的是，输入神经元和权重数据均需要声明为有符号数 (`signed`)，确保 Verilog 代码中使用的乘法运算符 (`*`) 能够正确生成有符号数乘法器。该设计采用固定流水线结构：输入数据采样与寄存在第一拍完成，乘积累加计算与结果输出在第二拍完成。虽然能够实现每个时钟周期处理向量长度为 4 的完整内积运算，但输出结果固定延迟两拍。由于该设计没有提供输出结果有效信号，这两拍延迟需要在 DLP 控制器中进行针对性的设计才能保证整个 DLP 时序衔接正确（保证便笺存储器在正确的时刻接收运算结果）。这简化了内积运算器本身的设计，却一定程度上给 DLP 整体的控制带来了额外的麻烦。

表 8.1 内积运算器输入输出接口信号

域	位宽	功能描述
<code>activations</code>	4×16	并行输入 INT16 神经元激活值
<code>weights</code>	4×16	并行输入 INT16 权重
<code>enable</code>	1	使能信号，高电平有效
<code>reset</code>	1	同步复位信号，高电平有效
<code>clk</code>	1	输入时钟信号
<code>result</code>	32	输出内积运算结果

我们可以向大语言模型提问，让它给我们提供一份 Verilog 代码。提问时，我们可以附加一些额外要求，例如要求限定运算的比特位宽；要求添加复位和使能信号；要求代码简

洁易懂、不要使用 Verilog 中复杂的高级功能等。例如，向大语言模型输入以下提示词：

用 Verilog 语言编写一个并行内积运算器，输入 4 个 16 位整数输入神经元激活值、4 个 16 位整数权重，计算它们的内积结果（32 位整数）。注意：

- 32 位整数已经几乎足够表达所有输出值范围，仅全部输入-32768 时会产生溢出，我们暂时可以忽略这种极端情况。
- 添加同步复位信号和使能信号，并在输入数据上添加寄存器。使能信号无效时，令输入寄存器保持锁定，避免乘法器和加法树电路发生翻转浪费能量。
- 尽量令代码简洁、易于理解。例如，使用 `signed` 关键字似乎比使用 `$signed` 运算符易读；使用 `genvar` 不如将代码展开易读。

DeepSeek-V3 在回答中提供了以下代码实现：

代码示例 8.1 内积运算器（第一版）

```

1 module inner_product_4x16 (
2     input wire clk,           // 时钟信号
3     input wire reset,        // 同步复位信号
4     input wire enable,       // 使能信号
5     input wire signed [15:0] activations [0:3], // 4个16位激活值
6     input wire signed [15:0] weights [0:3],    // 4个16位权重
7     output reg signed [31:0] result            // 32位结果
8 );
9 // 输入寄存器
10 reg signed [15:0] activations_reg [0:3];
11 reg signed [15:0] weights_reg [0:3];
12
13 // 部分乘积
14 wire signed [31:0] partial_products [0:3];
15
16 // 输入寄存器更新逻辑
17 integer i;
18 always @(posedge clk) begin
19     if (reset) begin
20         for (i = 0; i < 4; i = i + 1) begin
21             activations_reg[i] <= 16'd0;
22             weights_reg[i] <= 16'd0;
23         end
24     end else if (enable) begin
25         for (i = 0; i < 4; i = i + 1) begin
26             activations_reg[i] <= activations[i];
27             weights_reg[i] <= weights[i];
28         end
29     end
30 end
31
32 // 并行乘法
33 assign partial_products[0] = activations_reg[0] * weights_reg[0];
34 assign partial_products[1] = activations_reg[1] * weights_reg[1];
35 assign partial_products[2] = activations_reg[2] * weights_reg[2];
36 assign partial_products[3] = activations_reg[3] * weights_reg[3];
37
38 // 累加结果
39 always @(posedge clk) begin
40     if (reset) begin
41         result <= 32'd0;

```

```
42     end else if (enable) begin
43         result <= partial_products[0] + partial_products[1] +
44             partial_products[2] + partial_products[3];
45     end
46 end
47 endmodule
```

这份代码完全符合我们的要求。如果不确定，我们还可以进一步要求大语言模型给我们提供解释。我们向大语言模型继续输入以下提示词：

向我讲解这份代码。

DeepSeek-V3 提供了每一部分代码的详尽解释：

Verilog 并行内积运算器详细解析 我将详细讲解这个 4 元素并行内积运算器的 Verilog 实现，帮助你全面理解每个部分的设计原理和实现细节。

模块接口

```
1 module inner_product_4x16 (
2     input wire clk,           // 时钟信号
3     input wire reset,        // 同步复位信号
4     input wire enable,       // 使能信号
5     input wire signed [15:0] activations [0:3], // 4个16位激活值
6     input wire signed [15:0] weights [0:3],    // 4个16位权重
7     output reg signed [31:0] result            // 32位结果
8 );
```

- **时钟信号 (clk)**: 同步所有寄存器操作
- **复位信号 (reset)**: 同步复位所有寄存器
- **使能信号 (enable)**: 控制数据更新和计算
- **输入数组**: 4 个 16 位有符号激活值和权重
- **输出**: 32 位有符号内积结果

输入寄存器

```
1 reg signed [15:0] activations_reg [0:3];
2 reg signed [15:0] weights_reg [0:3];
```

这两组寄存器用于存储输入数据，实现流水线设计：

1. 在使能信号有效时捕获输入数据
2. 在使能信号无效时保持数据不变
3. 防止输入变化直接传播到乘法器，减少不必要的翻转

寄存器更新逻辑

```
1 always @(posedge clk) begin
2     if (reset) begin
3         for (i = 0; i < 4; i = i + 1) begin
4             activations_reg[i] <= 16'd0;
```



```

5         weights_reg[i] <= 16'd0;
6     end
7 end else if (enable) begin
8     for (i = 0; i < 4; i = i + 1) begin
9         activations_reg[i] <= activations[i];
10        weights_reg[i] <= weights[i];
11    end
12 end
13 end

```

- 复位时：将所有寄存器清零
- 使能有效时：更新寄存器内容
- 使能无效时：寄存器保持原值
- 使用 <= 非阻塞赋值确保同步更新

并行乘法

```

1 wire signed [31:0] partial_products [0:3];
2
3 assign partial_products[0] = activations_reg[0] * weights_reg[0];
4 assign partial_products[1] = activations_reg[1] * weights_reg[1];
5 assign partial_products[2] = activations_reg[2] * weights_reg[2];
6 assign partial_products[3] = activations_reg[3] * weights_reg[3];

```

- 4 个乘法器并行工作
- 每个乘法器计算一对激活值和权重的乘积
- 结果宽度为 32 位 (16 位 × 16 位乘法)
- 使用 signed 确保有符号数正确相乘

累加阶段

```

1 always @(posedge clk) begin
2     if (reset) begin
3         result <= 32'd0;
4     end else if (enable) begin
5         result <= partial_products[0] + partial_products[1] +
6                 partial_products[2] + partial_products[3];
7     end
8 end

```

- 在时钟上升沿更新结果
- 复位时清零结果寄存器
- 使能有效时将 4 个部分积相加
- 32 位结果足够容纳最大可能值 (除全-32768 情况)

关键设计特点

1. 节能设计：

- 使能信号控制数据流动
- 无效时寄存器保持，防止组合逻辑翻转
- 减少动态功耗

2. 时序控制:

- 所有操作同步于时钟
- 复位同步进行
- 两级流水线（输入寄存器 + 结果寄存器）

3. 数值安全:

- 明确使用有符号数运算
- 32 位输出防止大多数溢出情况
- 复位确保初始状态稳定

4. 代码可读性:

- 展开循环更直观
- 使用 `signed` 关键字而非 `$signed` 函数
- 清晰的信号命名

—— 时序行为分析 ——

1. 正常操作时序:

- T0: 输入出现在 `activations/weights`
- T1(时钟上升沿): 如果 `enable=1`, 输入被锁存到寄存器
- T2(时钟上升沿): 乘积累加结果出现在输出

2. 使能无效时序:

- 当 `enable=0` 时, 寄存器保持原值
- 乘法器和加法器输入不变, 无翻转

3. 复位时序:

- `reset=1` 时, 下一时钟沿清零所有寄存器
- 输出 `result` 在复位后第二个时钟沿变为 0

这个设计在面积、速度和功耗之间取得了良好平衡, 特别适合需要节能的神经网络加速应用。

关于设计和程序编写的细节问题, 如有不确定之处, 我们还可以进一步追问大语言模型。例如, 我们进一步向大语言模型提问, 并要求它修改代码:

为什么乘法逻辑写在 `assign` 语句中, 加法逻辑却写在了 `always` 语句中? 请将加法树改为 `assign` 语句实现, 让代码更简短。

DeepSeek-V3 按照我们的要求提供了修改后的代码, 保持代码的功能不变, 但不再依照内积运算器的各个处理阶段编写代码, 而是拆分为了组合逻辑部分和时序逻辑部分两部分实现, 使代码明显更加紧凑了。更新后的代码如下:

代码示例 8.2 内积运算器 (第二版)

```
1 module inner_product_4x16 (  
2     input wire clk,
```

```

3   input wire reset ,
4   input wire enable ,
5   input wire signed [15:0] activations [0:3],
6   input wire signed [15:0] weights [0:3],
7   output reg signed [31:0] result
8 );
9   // 输入寄存器
10  reg signed [15:0] activations_reg [0:3];
11  reg signed [15:0] weights_reg [0:3];
12
13  // 组合逻辑部分
14  wire signed [31:0] product =
15      (activations_reg[0] * weights_reg[0]) +
16      (activations_reg[1] * weights_reg[1]) +
17      (activations_reg[2] * weights_reg[2]) +
18      (activations_reg[3] * weights_reg[3]);
19
20  // 寄存器更新逻辑
21  integer i;
22  always @(posedge clk) begin
23      if (reset) begin
24          for (i = 0; i < 4; i = i + 1) begin
25              activations_reg[i] <= 16'd0;
26              weights_reg[i] <= 16'd0;
27          end
28          result <= 32'd0;
29      end else if (enable) begin
30          for (i = 0; i < 4; i = i + 1) begin
31              activations_reg[i] <= activations[i];
32              weights_reg[i] <= weights[i];
33          end
34          result <= product;
35      end
36  end
37 endmodule

```

请注意，大语言模型编写的 Verilog 代码有时会出错，有时会给出不符合要求的设计。在实验过程中，如果使用大语言模型协助编写 Verilog 代码，需要经过仔细核对确认无误后才能采纳。

8.5.2 矩阵乘向量运算器

本节实验将在内积运算器的基础上扩展设计矩阵乘向量运算器，实现每个周期完成 4×4 权重矩阵与 4 个输入神经元激活值的矩阵-向量乘法运算。图8.3展示了矩阵乘向量运算器的结构示例，基于之前设计的内积运算器，结构简单，仅并行连接四个内积运算器即可组成。该设计的输入接口同时接收尺寸为 4×4 的权重矩阵和 4 元素输入神经元激活值向量，其中输入神经元激活值向量通过广播机制同时传输到所有内积运算器。由于所有内积运算器共享了相同的输入神经元激活值向量，矩阵乘向量运算器所需的外部数据接口相对四个分离的内积运算器而言更少，因此运算密度更高，在同等便笺存储器访问带宽下提供了更高的算力。

矩阵乘向量运算器的接口信号定义如表8.3所示。核心信号包括系统时钟 `clk`、同步复位信号 `reset`（高电平有效）、计算使能信号 `enable`（高电平有效）。数据接口包含 64 位宽度的输入神经元激活值向量（`activations[0:3]`）、256 位宽度的输入权重矩阵

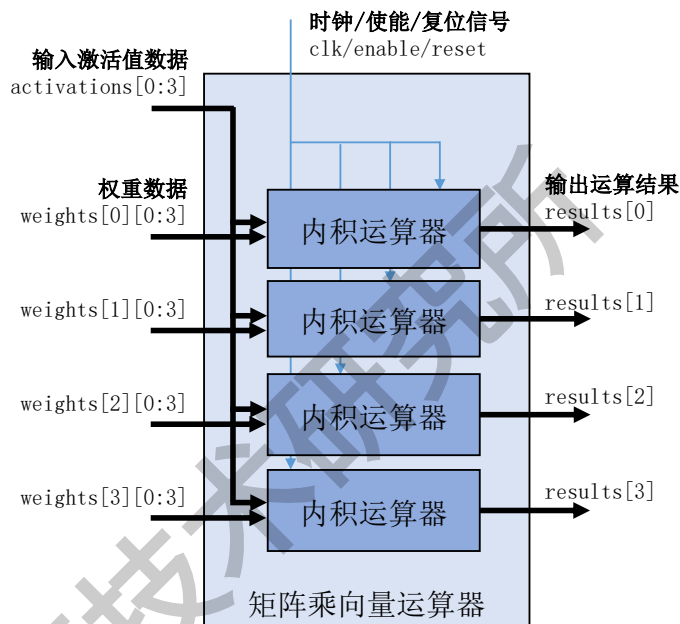


图 8.3 矩阵乘向量运算器

(weights[0:3][0:3]) 以及 128 位宽度的输出结果向量 (results[0:3])。所有数据均采用有符号表示法。

表 8.2 矩阵乘向量运算器输入输出接口信号

域	位宽	功能描述
activations	4×16	并行输入 INT16 神经元激活值向量
weights	4×4×16	并行输入 INT16 权重矩阵
enable	1	使能信号，高电平有效
reset	1	同步复位信号，高电平有效
clk	1	输入时钟信号
results	4×32	输出乘法运算结果向量

代码示例8.3展示了矩阵乘向量运算器的代码实现框架。通过 Verilog 语言实现矩阵乘向量运算器，需要通过实例化四个内积运算器完成。每个实例接收相同的时钟、复位、使能信号以及广播的输入神经元激活值向量，但分别接收权重矩阵的不同行。具体实现代码展示了一个顶层模块 matrix_vector_mult_4x4x16，内部实例化四个 inner_product_4x16 模块，将权重矩阵按行分配给各内积运算器，输出结果组合为向量。请尝试根据上述功能描述补全代码。

代码示例 8.3 矩阵乘向量运算器代码

```

1 module matrix_vector_mult_4x4x16 (
2     input wire clk,
3     input wire reset,
4     input wire enable,
5     input wire signed [15:0] activations [0:3],
6     input wire signed [15:0] weights [0:3][0:3],
7     output wire signed [31:0] results [0:3]
8 );

```

```

9 // 实例化4个内积运算器
10 inner_product_4x16 ipu0 (
11     .clk(clk),
12     .reset(reset),
13     .enable(enable),
14     -----
15     -----
16     -----
17 );
18 inner_product_4x16 ipu1 (
19     .clk(clk),
20     .reset(reset),
21     .enable(enable),
22     -----
23     -----
24     -----
25 );
26 inner_product_4x16 ipu2 (
27     .clk(clk),
28     .reset(reset),
29     .enable(enable),
30     -----
31     -----
32     -----
33 );
34 inner_product_4x16 ipu3 (
35     .clk(clk),
36     .reset(reset),
37     .enable(enable),
38     -----
39     -----
40     -----
41 );
42 endmodule

```

该设计继承内积运算器的两级流水线时序特性。第一拍完成输入神经元激活值向量和权重矩阵的锁存，第二拍四个内积运算器并行完成乘累加计算。在使能信号有效后的第二个时钟上升沿，结果向量将稳定输出。该设计每个时钟周期可完成一次完整的4阶方阵与长度为4的向量之间的乘法运算，但需要注意运算结果依然固定延迟2拍才产生。

8.5.3 矩阵乘法运算器

本节实验将在内积运算器、矩阵乘向量运算器的基础上继续扩展，设计矩阵乘法运算器，实现每个周期完成 4×4 权重矩阵与4组4元素输入神经元激活值的矩阵乘法运算。图8.4展示了矩阵乘法运算器的结构示例，采用四个矩阵乘向量运算器并行连接组成。该设计的输入接口同时接收尺寸为 4×4 的权重矩阵和尺寸为 4×4 的输入神经元激活值矩阵，其中每个矩阵乘向量运算器得到一组输入神经元激活值向量，而权重矩阵则通过广播机制同时传输到所有矩阵乘向量运算器。由于所有矩阵乘向量运算器共享了相同的权重矩阵，矩阵乘法运算器所需的外部数据接口相对分立的内积运算器、矩阵乘向量运算器而言进一步减少，因此运算密度进一步提高，在同等便笺存储器访问能力下所达到的理论性能更强。

然而，采用矩阵乘法运算器作为DLP中的矩阵运算单元，相比矩阵乘向量运算器也存在一些缺点。首先，在深度神经网络当中，全连接层在推理过程中对应的运算模式是矩阵乘向量，而要组成矩阵乘法运算，需要应用层面将多次推理组成一定量的批次一同执行，这

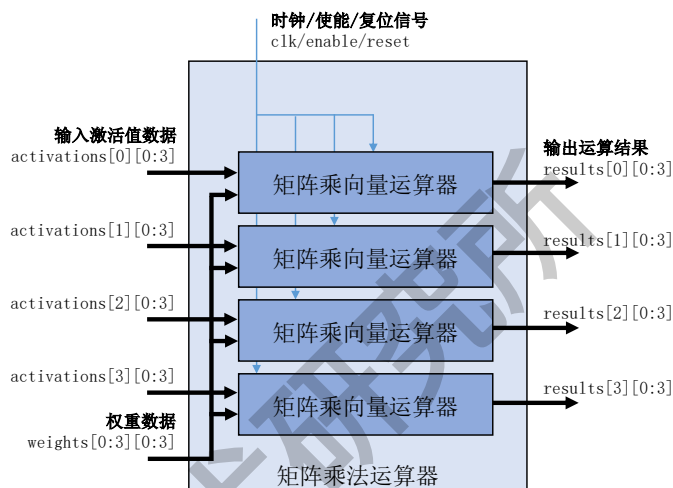


图 8.4 矩阵乘法运算器

对应用场景提出了额外的限制。其次，从电路角度考虑，由于矩阵乘法运算器规模进一步加大，而且权重、激活值都在各自层次经历过广播操作，这会导致电路中连线的距离变远、扇出加大，对电路信号传输的电气性能造成了一定的挑战，频率、功耗等指标更难满足设计要求。在本实验中，我们实现的还是一个非常小规模示例（仅处理 4 阶矩阵），目的是进行原理性的展示，实验验证功能正确即可，还未涉及到这些工程中会遇到的挑战。但读者应当充分理解 DLP 的设计取决于诸多方面的因素，不能简单地认为理论性能越强越好。

需要注意的是，此处矩阵乘法运算器的用途是进行深度学习运算，直接支持例如含批次的全连接层等关键算子。因此，此处的“矩阵乘法”与线性代数中的概念还存在细微的差别。在线性代数中，矩阵乘法规定左矩阵的行向量与右矩阵的列向量进行内积运算，即在 $Y = WX$ 的运算过程中，要求 $Y(i, j) = \sum_k W(i, k)X(k, j)$ ；而此处，权重矩阵中对应同一输出特征、不同输入特征的权重向量参与内积运算，相应地，输入神经元激活值矩阵中同一批次的激活值向量参与内积运算。即，矩阵乘法运算器的运算过程是 $results[j][i] = weights[i][0:3] \cdot activations[j][0:3]$ （“ $\cdot$ ”表示内积），在运算次序上两方矩阵都可以以行向量参与内积运算，而不需要先行矩阵转置。这是由便笺存储器只能按行访问的电路原理决定的实现细节差别，从数学本质上并未超出线性代数规定的矩阵乘法概念，因此我们仍以矩阵乘法称呼它。

矩阵乘法运算器的接口信号定义如表 8.3 所示。核心信号包括系统时钟 `clk`、同步复位信号 `reset`（高电平有效）、计算使能信号 `enable`（高电平有效）。数据接口包含 256 位宽度的输入神经元激活值矩阵（`activations[0:3][0:3]`）、256 位宽度的输入权重矩阵（`weights[0:3][0:3]`）以及 512 位宽度的输出结果向量（`results[0:3][0:3]`）。所有数据均采用有符号表示法。

代码示例 8.4 展示了矩阵乘法运算器的代码实现框架。通过 Verilog 语言实现矩阵乘法运算器，需要通过实例化四个矩阵乘向量运算器（或实例化十六个内积运算器）完成。每个矩阵乘向量运算器实例接收相同的时钟、复位、使能信号以及广播的权重矩阵，但分别接收输入神经元激活值不同批次的样本。具体实现代码展示了一个顶层模块 `matrix_mult_4x4x4x16`,

表 8.3 矩阵乘法运算器输入输出接口信号

域	位宽	功能描述
activations	4×4×16	并行输入 INT16 神经元激活值矩阵
weights	4×4×16	并行输入 INT16 权重矩阵
enable	1	使能信号，高电平有效
reset	1	同步复位信号，高电平有效
clk	1	输入时钟信号
results	4×4×32	输出乘法运算结果矩阵

内部实例化四个 `matrix_vector_mult_4x4x16` 模块，将输入神经元激活值矩阵按行分配给各矩阵乘向量运算器，输出结果组合为矩阵。请尝试根据上述功能描述补全代码。

代码示例 8.4 矩阵乘法运算器代码

```

1 module matrix_mult_4x4x16 (
2     input wire clk,
3     input wire reset,
4     input wire enable,
5     input wire signed [15:0] activations [0:3][0:3],
6     input wire signed [15:0] weights [0:3][0:3],
7     output wire signed [31:0] results [0:3][0:3]
8 );
9 // 实例化4个矩阵乘向量运算器
10 matrix_vector_mult_4x4x16 mxv0 (
11     .clk(clk),
12     .reset(reset),
13     .enable(enable),
14     -----
15     -----
16     -----
17 );
18 matrix_vector_mult_4x4x16 mxv1 (
19     .clk(clk),
20     .reset(reset),
21     .enable(enable),
22     -----
23     -----
24     -----
25 );
26 matrix_vector_mult_4x4x16 mxv2 (
27     .clk(clk),
28     .reset(reset),
29     .enable(enable),
30     -----
31     -----
32     -----
33 );
34 matrix_vector_mult_4x4x16 mxv3 (
35     .clk(clk),
36     .reset(reset),
37     .enable(enable),
38     -----
39     -----
40     -----
41 );
42 endmodule

```

同样地，该设计依然继承了内积运算器的两级流水线时序特性。第一拍完成输入神经

元激活值矩阵和权重矩阵的锁存，第二拍十六个内积运算器并行完成乘累加计算。在使能信号有效后的第二个时钟上升沿，结果矩阵将稳定输出。该设计每个时钟周期可完成一次完整的在两个4阶方矩阵之间的矩阵乘法运算，运算结果仍是固定延迟2拍才产生。

8.5.4 编译调试

8.5.4.1 编写仿真顶层文件

仿真顶层文件 `tb_top.cpp` 读取配置文件 `config`，输入到矩阵运算子单元的控制信号端口；并读取神经元数据文件 `neuron` 和权重数据文件 `weight`，输入到矩阵运算子单元的神经元端口和权重端口，同时也需要将结果与标准结果文件 `result` 进行比较。

在 C++ 中，通常使用文件流操作从文件中读取数据到存储器中，如代码示例8.5所示。

代码示例 8.5 读取配置文件和数据

```
1 // 读取配置文件（向量维度和位宽）
2 std::ifstream config_file("../data/config");
3 // 读取神经元和权重数据（十六进制）
4 std::ifstream neuron_file("../data/neuron");
5 std::ifstream weight_file("../data/weight");
```

8.5.4.2 编译代码

仿真顶层文件编写完成后，需要配置编译列表文件 `compile`，用于指定需要编译的 Verilog 源文件路径，当项目包含多个模块时，可以通过配置编译列表文件一次性导入所有依赖文件，避免在命令行中手动输入大量源文件路径，简化编译流程。编译列表指定需编译的顶层文件和源代码文件，代码示例如8.6所示。

代码示例 8.6 编译列表

```
1 # 内积运算器源码路径
2 ../src/inner_product_4x16.v
```

然后，配置 Makefile 文件，Makefile 是一个自动化构建脚本，用于定义项目的编译规则和依赖关系。在硬件仿真中，它通常用来调用 Verilog 仿真工具（如 VCS、Verilator）。代码示例如8.7所示。

代码示例 8.7 编译脚本

```
1 # 工具配置
2 VERILATOR := verilator
3 VERILATOR_FLAGS := --cc --exe --build -Wall --trace # 生成C++模型+波形
4 CXX := g++
5 CXXFLAGS := -std=c++14 # C++标准
6
7 # 文件列表
8 VERILOG_SRCS := $(shell cat compile.f) # 从compile.f读取Verilog文件
9 TESTBENCH := tb_top.cpp # C++测试平台
10
11 # 编译目标：生成可执行仿真文件
12 all:
13     @echo "=== 开始编译 ==="
```



```

14 $(VERILATOR) $(VERILATOR_FLAGS) \
15     --top-module inner_product_4x16 \ # 顶层 Verilog 模块
16     $(VERILOG_SRCS) \                # Verilog 源码
17     $(TESTBENCH) \                  # C++ 测试平台
18     -CFLAGS "$(CXXFLAGS)" \         # C++ 编译参数
19     -o sim_executable                # 输出文件名
20
21 # 清理编译产物
22 clean:
23     rm -rf obj_dir/ sim_executable waveform.vcd
24     @echo "=== 清理完成 ==="
25
26 # 伪目标（避免与文件重名）
27 .PHONY: all clean

```

完成上述操作后，在命令窗口输入“**make**”命令开始编译，Verilator 编译代码时，命令行窗口中将显示编译的文件、执行的编译指令、编译状态等信息。编译顺利完成后会产生可执行文件：**waveform.vcd**（位于当前目录下）和中间文件目录：**obj_dir**（包含 Verilator 生成的 C++ 模型和编译产物）。

8.5.4.3 启动仿真

在命令行窗口输入“**gtkwave waveform.vcd &**”命令启动 Verilator 生成的仿真可执行文件，从而触发 GTKWave 实现波形仿真。Verilator 运行执行脚本后将显示如图8.5所示的仿真实例面板。

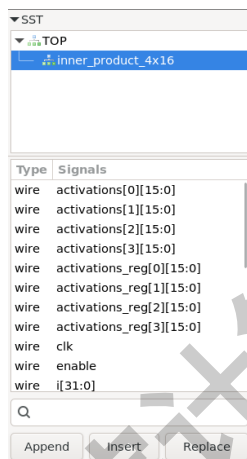


图 8.5 仿真实例面板

8.5.4.4 添加观测信号

在仿真实例面板的“SST”面板中，先选中要调试的模块，然后右键单击“Recurse import -> append”，可以将所有信号添加到“Wave”窗口，也可以在选中模块后，于下方信号列表中进行选择性添加。

8.5.4.5 验证

根据“Wave”窗口显示的信号波形，观察信号波形是否符合预期，主要观察时钟信号工作时，是否呈现周期性翻转；复位信号值为 1 时输入输出值是否全为 0；两个输入信号所显示的值与输入文件 `data/neuron`、`data/weight` 的值是否一致；输出值 `result` 与输入值进行相应运算得到的最终结果是否一致。

8.5.4.6 迭代调试

当发现仿真结果错误后，重新修改代码，执行以下步骤：

- 1) 运行“`make clean`”清理旧的编译产物；
- 2) 运行“`make`”命令重新编译；
- 3) 执行仿真可执行文件启动 `GTKWave`，执行仿真观察结果；
- 4) 仿真完成后，退出仿真。

8.6 实验评估

实现内积运算器、矩阵乘向量运算器及矩阵乘法运算器，再通过 Verilator 对 4 组不同规模矩阵的运算分别仿真。运算结果与 `result` 文件中的结果均比对正确，表明所实现运算器功能正确。

本次实验的评估标准设定如下：

- 60 分：完成内积运算器，输出结果和 `result` 文件比对正确。
- 100 分：完成矩阵乘向量运算器，输出结果和 `result` 文件比对正确。
- 120 分：完成内积运算器、矩阵乘向量运算器和矩阵乘法运算器，输出结果和 `result` 文件比对正确。

8.7 实验思考

1) 对比分析内积运算器、矩阵乘向量运算器和矩阵乘法运算器内部的乘法器数量、加法器数量和对外数据接口数量，计算并对比每种设计中这些元素的比例。如果增大三种运算器的规模（例如，向量长度从 4 增加到 16 乃至 128），这些设计的比例分别会发生什么样的变化？

2) 请说明深度学习处理器加速深度学习计算的基本原理是什么？